
BÁO CÁO PHƯƠNG TRÌNH SCHRÖDINGER

PHỤ THUỘC THỜI GIAN

PHƯƠNG PHÁP RUNGE-KUTTA BẬC 4 (RK4)

SAI PHÂN KHÔNG GIAN BẬC 4 – DẠNG MA TRẬN TIỀN HÓA

Họ và tên: Nguyễn Trần Gia Bảo

MSSV: 23137003

Môn học: Vật lý tính toán (PHY10532)

Giảng viên: TS. Vũ Quang Tuyên

Ngày 13 tháng 6 năm 2026

Mục lục

1	Giới thiệu và nội dung tính toán	2
2	Cơ sở lý thuyết	2
2.1	Phương trình Schrödinger phụ thuộc thời gian	2
2.2	Bó sóng Gauss ban đầu	3
2.3	Lưới sai phân và sai phân không gian bậc 4	3
2.4	Dạng ma trận của TDSE	3
2.5	Phương pháp Runge-Kutta bậc 4	4
2.6	Điều kiện Courant-Friedrichs-Lewy (CFL)	5
3	Tham số và mô hình rời rạc trong code	5
3.1	Tham số bài toán	5
3.2	Thế năng và điều kiện biên	5
4	Phân tích chương trình Python	6
4.1	Gọi thư viện và khởi tạo bó sóng Gauss	6
4.2	Tạo lưới không gian và thời gian	7
4.3	Kiểm tra điều kiện CFL	7
4.4	Xây dựng ma trận tiến hóa M	7
4.5	Giải TDSE bằng RK4	9
4.6	Ghi hàm sóng ra tập tin văn bản	10
4.7	Tính và ghi chuẩn xác suất	11
4.8	Gán tham số, chạy hàm giải và lưu kết quả	12
4.9	Đọc dữ liệu để vẽ hình	12
4.10	Vẽ và lưu đồ thị bảo toàn xác suất	13
4.11	Tạo các giá trị σ_0 và k_0	13
5	Kết quả và thảo luận	14
5.1	So sánh Leapfrog bậc 4 và RK4	14
5.2	Bảo toàn xác suất	15
5.3	Sóng truyền ngược khi $k_0 = -10$	16
5.4	Ảnh hưởng của độ lớn k_0 đến vận tốc và phản xạ	17
5.5	Ảnh hưởng của độ rộng ban đầu σ_0	18
6	Kết luận	20

1. Giới thiệu và nội dung tính toán

Nghiệm số của phương trình Schrödinger phụ thuộc thời gian một chiều được giải bằng phương pháp *Runge-Kutta bậc 4* (RK4). Đạo hàm bậc hai theo không gian được rời rạc hóa bằng sai phân trung tâm năm điểm (bậc 4), đưa TDSE về dạng một hệ phương trình vi phân thường (ODE) tuyến tính

$$\frac{d\psi}{dt} = M\psi,$$

với M là một ma trận vuông không phụ thuộc thời gian. Hệ ODE này sau đó được tiến theo thời gian bằng công thức RK4.

Chương trình Python sẽ chạy các bước tính toán như sau:

1. Khởi tạo bó sóng Gauss chuẩn hóa tại $x_0 = 5$ với độ rộng σ_0 và số sóng trung tâm k_0 .
2. Xây dựng lưới đều theo x và t , kiểm tra điều kiện Courant-Friedrichs-Lewy (CFL) trước khi giải.
3. Xây dựng ma trận tiến hóa M từ sai phân không gian bậc 4 và thế năng $V(x)$.
4. Giải hệ $d\psi/dt = M\psi$ bằng RK4, áp điều kiện biên Dirichlet tại hai đầu miền.
5. Ghi $\text{Re } \psi$, $\text{Im } \psi$, $|\psi|$ và $|\psi|^2$ ra file data.
6. Tính chuẩn xác suất $N(t) = \int |\psi(x, t)|^2 dx$ bằng quy tắc hình thang.
7. So sánh kết quả RK4 với kết quả Leapfrog bậc 4 tại $\sigma_0 = 0.5$, $k_0 = 8$.
8. Khảo sát chiều truyền, vận tốc nhóm và phản xạ của bó sóng khi thay đổi k_0 ; chương trình đồng thời quét $\sigma_0 \in \{0.5, 1, 2\}$ và $k_0 \in \{2, 8, 10, -2, -8, -10\}$.

2. Cơ sở lý thuyết

2.1. Phương trình Schrödinger phụ thuộc thời gian

Trong một chiều, phương trình Schrödinger phụ thuộc thời gian có dạng

$$i\hbar \frac{\partial \psi(x, t)}{\partial t} = \hat{H}\psi(x, t) = \left[-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right] \psi(x, t). \quad (1)$$

Mật độ xác suất được xác định bởi

$$\rho(x, t) = |\psi(x, t)|^2, \quad (2)$$

và điều kiện chuẩn hóa là

$$N(t) = \int_{-\infty}^{+\infty} |\psi(x, t)|^2 dx = 1. \quad (3)$$

Trong mô phỏng hữu hạn, tích phân được lấy trên miền $[x_{\min}, x_{\max}]$.

2.2. Bó sóng Gauss ban đầu

Bó sóng chuẩn hóa được sử dụng trong bài toán này

$$\psi(x, 0) = \frac{1}{(\pi\sigma_0^2)^{1/4}} \exp\left[-\frac{1}{2}\left(\frac{x-x_0}{\sigma_0}\right)^2\right] \exp(ik_0x). \quad (4)$$

Hệ số $(\pi\sigma_0^2)^{-1/4}$ làm cho bó sóng được chuẩn hóa trên toàn trục số. Với $x_0 = 5$ và $\sigma_0 = 0.5$, phần đuôi của bó sóng tại hai biên $x = 0$ và $x = 15$ ban đầu rất nhỏ, nên tích phân trên miền mô phỏng xấp xỉ 1.

Nếu V không phụ thuộc x trong miền chuyển động, quan hệ tán sắc liên tục là

$$E(k) = \frac{\hbar^2 k^2}{2m} + V, \quad (5)$$

và vận tốc nhóm của tâm bó sóng là

$$v_g = \frac{1}{\hbar} \frac{dE}{dk} = \frac{\hbar k_0}{m}. \quad (6)$$

Với quy ước $\hbar = m = 1$ trong chương trình, $v_g = k_0$. Vì vậy dấu của k_0 xác định chiều truyền, còn độ lớn $|k_0|$ xác định tốc độ chuyển động của tâm bó sóng trước khi phản xạ ở biên.

2.3. Lưới sai phân và sai phân không gian bậc 4

Miền được rời rạc hóa theo

$$x_j = x_{\min} + j\Delta x, \quad t_n = t_{\min} + n\Delta t, \quad \psi_j(t_n) = \psi(x_j, t_n). \quad (7)$$

Đạo hàm bậc hai theo không gian được xấp xỉ bằng sai phân trung tâm năm điểm

$$\left. \frac{\partial^2 \psi}{\partial x^2} \right|_j = \frac{-\psi_{j+2} + 16\psi_{j+1} - 30\psi_j + 16\psi_{j-1} - \psi_{j-2}}{12\Delta x^2} + \mathcal{O}(\Delta x^4). \quad (8)$$

Sai phân này có cùng bậc chính xác không gian ($\mathcal{O}(\Delta x^4)$) với phương pháp Leapfrog bậc 4 đã trình bày trước đó; điểm khác biệt giữa hai phương pháp nằm hoàn toàn ở cách rời rạc hóa *thời gian*.

2.4. Dạng ma trận của TDSE

Thay xấp xỉ (8) vào (1) và sử dụng $\hbar = m = 1$, toán tử Hamilton tác dụng lên ψ_j trở thành

$$(\hat{H}\psi)_j = \left(\frac{5}{4\Delta x^2} + V_j \right) \psi_j - \frac{16}{24\Delta x^2} (\psi_{j+1} + \psi_{j-1}) + \frac{1}{24\Delta x^2} (\psi_{j+2} + \psi_{j-2}). \quad (9)$$

Từ (1), $d\psi/dt = -i\hat{H}\psi/\hbar = -i\hat{H}\psi$. Đặt

$$g_j = -i \left(\frac{5}{4\Delta x^2} + V_j \right), \quad a = \frac{i}{24\Delta x^2}, \quad (10)$$

ta thu được hệ ODE tuyến tính cho từng nút lưới

$$\boxed{\frac{d\psi_j}{dt} = g_j\psi_j + 16a(\psi_{j+1} + \psi_{j-1}) - a(\psi_{j+2} + \psi_{j-2})}. \quad (11)$$

Viết dưới dạng ma trận, $d\boldsymbol{\psi}/dt = M\boldsymbol{\psi}$, trong đó M là ma trận $N_x \times N_x$ có phần tử trên đường chéo chính bằng g_j , hai đường chéo lân cận ± 1 bằng $16a$, hai đường chéo lân cận ± 2 bằng $-a$, và tất cả các phần tử khác bằng 0. Cụ thể, hệ (11) tương đương với

$$\begin{pmatrix} \psi'_0 \\ \psi'_1 \\ \psi'_2 \\ \vdots \\ \psi'_{N_x-2} \\ \psi'_{N_x-1} \end{pmatrix} = \underbrace{\begin{pmatrix} g_0 & 16a & -a & 0 & \cdots & \cdots & 0 \\ 16a & g_1 & 16a & -a & \cdots & \cdots & 0 \\ -a & 16a & g_2 & 16a & -a & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & -a & 16a & g_{N_x-3} & 16a & -a \\ \vdots & \cdots & \ddots & -a & 16a & g_{N_x-2} & 16a \\ 0 & \cdots & \cdots & 0 & -a & 16a & g_{N_x-1} \end{pmatrix}}_M \begin{pmatrix} \psi_0 \\ \psi_1 \\ \psi_2 \\ \vdots \\ \psi_{N_x-2} \\ \psi_{N_x-1} \end{pmatrix}. \quad (12)$$

Đây chính là cấu trúc ma trận *băng* (banded) năm đường chéo: một đường chéo chính (hệ số g_j), hai đường chéo kề ± 1 (hệ số $16a$) và hai đường chéo kề ± 2 (hệ số $-a$). Trong code, hàng đầu, hàng cuối, cột đầu và cột cuối của ma trận (12) còn được gán lại bằng 0 để áp điều kiện biên Dirichlet, như sẽ trình bày ở mục 3.2 và mục 4.5. Ma trận M chỉ cần được xây dựng một lần vì không phụ thuộc thời gian.

2.5. Phương pháp Runge-Kutta bậc 4

Với hệ ODE $d\boldsymbol{\psi}/dt = f(\boldsymbol{\psi}) = M\boldsymbol{\psi}$, công thức RK4 cổ điển tính bốn hệ số góc trung gian

$$k_1 = M\boldsymbol{\psi}^n, \quad (13)$$

$$k_2 = M \left(\boldsymbol{\psi}^n + \frac{\Delta t}{2} k_1 \right), \quad (14)$$

$$k_3 = M \left(\boldsymbol{\psi}^n + \frac{\Delta t}{2} k_2 \right), \quad (15)$$

$$k_4 = M (\boldsymbol{\psi}^n + \Delta t k_3), \quad (16)$$

rồi cập nhật nghiệm theo

$$\boxed{\boldsymbol{\psi}^{n+1} = \boldsymbol{\psi}^n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)}. \quad (17)$$

Với M là ma trận hằng, công thức (17) tương đương với khai triển Taylor của toán tử tiến hóa $e^{\Delta t M}$ đến bậc 4:

$$\boldsymbol{\psi}^{n+1} \approx \left(I + \Delta t M + \frac{(\Delta t M)^2}{2} + \frac{(\Delta t M)^3}{6} + \frac{(\Delta t M)^4}{24} \right) \boldsymbol{\psi}^n, \quad (18)$$

sai số cục bộ là $\mathcal{O}(\Delta t^5)$ và sai số toàn cục là $\mathcal{O}(\Delta t^4)$. Khác với Leapfrog – một công thức ba mức thời gian với sai số toàn cục $\mathcal{O}(\Delta t^2)$ và cần một bước Euler riêng để khởi động – RK4 là một phương pháp một bước (chỉ cần $\boldsymbol{\psi}^n$ để tính $\boldsymbol{\psi}^{n+1}$), nên không cần công thức khởi động đặc biệt và có bậc chính xác thời gian cao hơn.

2.6. Điều kiện Courant-Friedrichs-Lewy (CFL)

Phương pháp tường minh chỉ ổn định (hàm sóng không bị vô hạn) nếu thỏa mãn điều kiện Courant-Friedrichs-Lewy (CFL):

$$\Delta t \leq \frac{1}{2/\Delta x^2 + V_{\max}}, \quad (19)$$

ứng với $\hbar = m = 1$. Với

$$\Delta x = 0.05, \quad \Delta t = \frac{1}{4}\Delta x^2 = 6.25 \times 10^{-4}, \quad V_{\max} = 500,$$

giới hạn trong code là

$$\Delta t_{\max} = \frac{1}{2/(0.05)^2 + 500} = 7.6923 \times 10^{-4}. \quad (20)$$

Do $\Delta t < \Delta t_{\max}$ nên điều kiện CFL được thỏa mãn.

3. Tham số và mô hình rời rạc trong code

3.1. Tham số bài toán

Đại lượng	Giá trị
$x_{\min}, x_{\max}$	0, 15
$t_{\min}, t_{\max}$	0, 1.2
Δx	0.05
Δt	$0.25\Delta x^2 = 0.000625$
N_x	301
N_t	1921
x_0	5
σ_0 trong phép so sánh chính	0.5
k_0 trong phép so sánh chính	8
$\hbar, m$	1, 1
$V_{\max}$	500
Chu kỳ ghi dữ liệu theo thời gian	100 bước, tương ứng $\Delta t_{\text{save}} = 0.0625$

3.2. Thế năng và điều kiện biên

Hàm thế năng được đặt như sau:

```

1  def thenang(x):
2      global V_max
3
4      if 0 <= x <= 15:
5          return 0
6      else:
7          return V_max

```

Hàm trên mô tả thế năng

$$V(x) = \begin{cases} 0, & 0 \leq x \leq 15, \\ V_{\max}, & x < 0 \text{ hoặc } x > 15. \end{cases} \quad (21)$$

Trong chương trình, lưới không gian chỉ được xây dựng trên miền $0 \leq x \leq 15$. Vì vậy, tại mọi điểm lưới được sử dụng trong phép tính, hàm `thenang(x)` đều trả về $V(x) = 0$. Giá trị $V_{\max}$ chỉ được gán cho vùng nằm ngoài miền mô phỏng và không trực tiếp xuất hiện trong ma trận M được sử dụng.

Hai biên của miền được xem như các giếng thế vuông, tương ứng với điều kiện Dirichlet

$$\psi(0, t) = \psi(15, t) = 0. \quad (22)$$

Khác với cách lập vòng lặp `for j in range(...)` của Leapfrog, điều kiện biên trong RK4 được áp đặt trực tiếp lên ma trận M và lên nghiệm:

- Hai hàng $M[0, :]$ và $M[-1, :]$ được gán bằng 0, nên $d\psi_0/dt = d\psi_{N_x-1}/dt = 0$ tại mọi thời điểm.
- Hai cột $M[:, 0]$ và $M[:, -1]$ được gán bằng 0, để giá trị tại hai nút biên không truyền ảnh hưởng sang các nút lân cận thông qua M .
- Điều kiện ban đầu được đặt $\psi_0(0) = \psi_{N_x-1}(0) = 0$, và sau mỗi bước RK4, ψ_0 và ψ_{N_x-1} tiếp tục được gán lại bằng 0.

Kết hợp ba điều này, hai nút biên luôn bằng 0 trong suốt mô phỏng, tương đương với điều kiện Dirichlet (22).

Một điểm khác biệt so với Leapfrog bậc 4 cần lưu ý: do các đường chéo phụ ở khoảng cách ± 2 của M được xây dựng với độ dài $N_x - 2$ (không kéo dài ra ngoài miền), các nút $j = 1$ và $j = N_x - 2$ vẫn được cập nhật bằng phương trình (11) với số hạng ψ_{j-2} hoặc ψ_{j+2} nằm ngoài lưới bị bỏ qua (coi như bằng 0). Trong khi đó, ở Leapfrog bậc 4, vòng lặp `for j in range(2, N_x-2)` khiến hai nút này không được cập nhật và giữ nguyên giá trị 0. Vì vậy, miền động lực học thực sự của RK4 rộng hơn Leapfrog bậc 4 một nút lưới ở mỗi phía, dù cả hai cùng dùng sai phân không gian bậc 4.

4. Phân tích chương trình Python

4.1. Gọi thư viện và khởi tạo bó sóng Gauss

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def khoitao_psi0(x):
5     global sigma_0, k_0, x_0
6     psi_0 = np.exp(-1/2 * ((x-x_0)/sigma_0)**2) * np.exp(1j*k_0*x) *
7         ↪ (np.pi*sigma_0**2)**(-1/4)
8     return psi_0
```

Hai thư viện được nạp ngay đầu chương trình. NumPy cung cấp mảng số phức, phép nhân ma trận–vector, hàm mũ, phép lấy modulus, tích phân hình thang và đọc dữ liệu; Matplotlib được dùng ở phần vẽ chuẩn xác suất.

Hàm `khoitao_psi0(x)` nhận toàn bộ lưới không gian dưới dạng một mảng NumPy. Các phép toán trong biểu thức đều được thực hiện theo từng phần tử, nên kết quả `psi_0` là một vector phức có cùng kích thước với `x`. Thừa số

$$\exp\left[-\frac{1}{2}\left(\frac{x-x_0}{\sigma_0}\right)^2\right]$$

tạo đường bao Gauss, `np.exp(1j*k_0*x)` tạo pha sóng phẳng, còn `(np.pi*sigma_0**2)**(-1/4)` là hệ số chuẩn hóa.

4.2. Tạo lưới không gian và thời gian

```
1 def khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt):
2     N_x = int(round((x_max - x_min) / dx)) + 1
3     N_t = int(round((t_max - t_min) / dt)) + 1
4     x = np.linspace(x_min, x_max, N_x)
5     t = np.linspace(t_min, t_max, N_t)
6     return x, t, N_x, N_t
```

Số nút được tính từ độ dài miền và bước lưới. Phép `round` dùng để làm tròn, còn phép cộng 1 bảo đảm tính cả hai đầu mút. Sau đó `np.linspace` tạo các mảng `x` và `t` chứa chính xác N_x và N_t phần tử. Hàm trả đồng thời bốn đối tượng để bộ giải RK4 có thể dùng trực tiếp lưới và kích thước mảng. Với bộ tham số chính, kết quả là $N_x = 301$ và $N_t = 1921$.

4.3. Kiểm tra điều kiện CFL

```
1 def kiểmtra_CFL(dt, dx, V_max):
2     dieu_kien = 1 / (2 / dx**2 + V_max)
3     if dt <= dieu_kien:
4         print("Dieu kien CFL duoc thoa man.")
5         return True
6     else:
7         print("Khong the hoi tu do vi pham dieu kien CFL.")
8         return False
```

Biến `dieu_kien` chứa giới hạn

$$\Delta t_{\max} = \frac{1}{2/\Delta x^2 + V_{\max}}.$$

Hàm so sánh `dt` với giới hạn này, in thông báo tương ứng và trả về `True` hoặc `False`. Với $\Delta x = 0.05$, $V_{\max} = 500$ và $\Delta t = 0.000625$, nhánh `True` được thực hiện.

4.4. Xây dựng ma trận tiến hóa M

```
1 def khoitao_matran_M():
2     global N_x, N_t, x_min, x_max, t_min, t_max, dx, dt
3     g = np.zeros(N_x, dtype=complex)
4     a = np.zeros(N_x, dtype=complex)
```



```

5     for i in range(N_x):
6         x_i = x_min + i*dx
7         V = thenang(x_i)
8         g[i] = -5j/(4*dx**2) - 1j*V
9         a[i] = 1j/(24*dx**2)
10    diag = np.diag(g)
11    off_diag1 = np.diag([16*a[i]]*(N_x-1), 1) + np.diag([16*a[i]]*(N_x-1),
12    ↪ -1)
13    off_diag2 = np.diag([-a[i]]*(N_x-2), 2) + np.diag([-a[i]]*(N_x-2), -2)
14    M = diag + off_diag1 + off_diag2
15
16    M[0, :] = 0
17    M[-1, :] = 0
18    M[:, 0] = 0
19    M[:, -1] = 0
20    return M

```

Hai mảng g và a có N_x phần tử phức, được tính theo (10): $g[i]$ ứng với $g_j = -i(5/(4\Delta x^2) + V_j)$, còn $a[i]$ ứng với $a = i/(24\Delta x^2) -$ tuy được lưu thành mảng, a thực chất là một hằng số vì V không xuất hiện trong biểu thức của nó.

Ma trận M được lắp ráp từ ba thành phần: **diag** là đường chéo chính chứa g_j ; **off_diag1** là hai đường chéo phụ ở vị trí ± 1 , mỗi phần tử bằng $16a$, ứng với số hạng $\psi_{j\pm 1}$ trong (11); **off_diag2** là hai đường chéo phụ ở vị trí ± 2 , mỗi phần tử bằng $-a$, ứng với số hạng $\psi_{j\pm 2}$. Lệnh `np.diag(..., k)` tạo một ma trận chỉ có đường chéo thứ k khác 0, nên tổng ba ma trận này cho đúng cấu trúc băng (banded) của (11) và (12).

Giải thích chi tiết lệnh np.diag. Hàm `np.diag(v, k)` của NumPy nhận một mảng/list v và đặt nó vào đường chéo thứ k của một ma trận vuông, các phần tử còn lại bằng 0:

- Tham số thứ nhất v là đoạn list chứa giá trị cần đặt. Để đặt được vào đường chéo thứ k ($|k| \geq 0$) của ma trận $N_x \times N_x$, độ dài của v phải là $N_x - |k|$.
- Tham số thứ hai k xác định vị trí đường chéo: $k = 0$ là đường chéo chính; $k = 1$ (hoặc $k = 2$) là đường chéo phụ nằm ngay phía *trên* đường chéo chính (superdiagonal); $k = -1$ (hoặc $k = -2$) là đường chéo phụ nằm ngay phía *dưới* đường chéo chính (subdiagonal).

Cụ thể với các dòng lệnh trong `khoitao_matran_M()`:

- `[16*a[i]]*(N_x-1)`: phép nhân một list có một phần tử với số nguyên $N_x - 1$ trong Python không phải là phép nhân số học, mà là phép *lặp lại danh sách* $N_x - 1$ lần. Vì `16*a[i]` là một số phức (do $a[i]$ là số phức), kết quả là một list gồm $N_x - 1$ phần tử, tất cả đều bằng $16a$ – đúng độ dài cần thiết cho đường chéo phụ thứ ± 1 của ma trận $N_x \times N_x$.
- `np.diag([16*a[i]]*(N_x-1), 1)`: đặt $N_x - 1$ giá trị $16a$ vào đường chéo ngay phía **trên** đường chéo chính, tức các vị trí $(j, j + 1)$ – ứng với hệ số của ψ_{j+1} .
- `np.diag([16*a[i]]*(N_x-1), -1)`: đặt $N_x - 1$ giá trị $16a$ vào đường chéo ngay phía **dưới** đường chéo chính, tức các vị trí $(j, j - 1)$ – ứng với hệ số của ψ_{j-1} . Tổng hai ma trận này tạo thành **off_diag1**.

- `np.diag([-a[i]]*(N_x-2), 2)` và `np.diag([-a[i]]*(N_x-2), -2)`: tương tự, nhưng đặt $N_x - 2$ giá trị $-a$ vào hai đường chéo cách đường chéo chính 2 bước, tức các vị trí $(j, j + 2)$ và $(j, j - 2)$ – ứng với hệ số của ψ_{j+2} và ψ_{j-2} . Tổng hai ma trận này tạo thành `off_diag2`. Vì đường chéo cách 2 bước luôn ngắn hơn đường chéo chính 2 phần tử, độ dài list tương ứng là $N_x - 2$ (không phải $N_x - 1$).

Cộng `diag`, `off_diag1` và `off_diag2` cho đúng ma trận bằng năm đường chéo ở (12).

Bốn lệnh cuối gán bằng 0 cho hàng đầu, hàng cuối, cột đầu và cột cuối của M , thực hiện điều kiện biên Dirichlet như đã trình bày ở mục 3.2. Ma trận M chỉ cần xây dựng một lần và được dùng lại cho mọi bước thời gian vì không phụ thuộc t .

4.5. Giải TDSE bằng RK4

```

1 def giaipt_psi_RK4():
2     global N_x, N_t, x_min, x_max, t_min, t_max, dx, dt
3
4     x, t, N_x, N_t = khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt)
5
6     y = np.zeros((N_t, N_x), dtype=np.complex128)
7
8     M = khoitao_matran_M()
9     psi_0 = khoitao_psi0(x)
10
11     psi_0[0] = 0.0
12     psi_0[-1] = 0.0
13
14     y[0, :] = psi_0
15
16     for n in range(N_t - 1):
17         k1 = M @ y[n]
18         k2 = M @ (y[n] + 0.5 * dt * k1)
19         k3 = M @ (y[n] + 0.5 * dt * k2)
20         k4 = M @ (y[n] + dt * k3)
21
22         y[n + 1] = y[n] + dt / 6.0 * (
23             k1 + 2.0 * k2 + 2.0 * k3 + k4
24         )
25
26         y[n + 1, 0] = 0.0
27         y[n + 1, -1] = 0.0
28
29     return y

```

Mảng `y` có dạng `[N_t, N_x]`, nghĩa là chỉ số thứ nhất biểu diễn thời gian và chỉ số thứ hai biểu diễn không gian, tương tự cấu trúc dùng trong bộ giải Leapfrog. Ma trận M được xây dựng một lần bằng `khoitao_matran_M()`, còn `psi_0` là bó sóng Gauss tại $t = 0$ với hai giá trị biên được gán lại bằng 0 trước khi gán cho `y[0, :]`.

Vòng lặp theo thời gian thực hiện đúng bốn bước của công thức RK4 (17): k_1 , k_2 , k_3 , k_4 lần lượt là $M\psi^n$, $M(\psi^n + \frac{1}{2}\Delta t k_1)$, $M(\psi^n + \frac{1}{2}\Delta t k_2)$ và $M(\psi^n + \Delta t k_3)$, mỗi bước là một phép nhân ma trận–vector (`@`) trên toàn bộ lưới. Nghiệm mới `y[n+1]` được tính bằng tổ

hợp $k_1 + 2k_2 + 2k_3 + k_4$ theo trọng số $\Delta t/6$. Sau mỗi bước, hai phần tử biên $y[n+1, 0]$ và $y[n+1, -1]$ được gán lại bằng 0 để đảm bảo điều kiện Dirichlet không bị trôi do sai số số học.

So với Leapfrog, bộ giải này không cần bước khởi động Euler riêng (vì RK4 là phương pháp một bước) và toàn bộ phép cập nhật không gian được thực hiện qua một phép nhân ma trận duy nhất thay vì vòng lặp `for j`.

4.6. Ghi hàm sóng ra tập tin văn bản

```

1 def luu_ketqua_psi(output_name, x, t, y, save_every_time=1, save_every_x =
  ↳ 1):
2     global N_x, N_t, dx, dt, sigma_0, k_0, x_min, x_max, t_min, t_max
3     filename = f"TDSE-ketqua-{output_name}.txt"
4
5     with open(filename, "w", encoding="utf-8") as f:
6         f.write("# KET QUA GIAI PHUONG TRINH SCHRODINGER PHU THUOC THOI
  ↳ GIAN\n\n")
7
8         f.write("#" * 150 + "\n")
9         f.write("# THAM SO DAU VAO\n")
10        f.write("#" * 150 + "\n\n")
11
12        f.write(f"# N_x      = {N_x}\n")
13        f.write(f"# N_t      = {N_t}\n")
14        f.write(f"# x_min    = {x_min}\n")
15        f.write(f"# x_max    = {x_max}\n")
16        f.write(f"# t_min    = {t_min}\n")
17        f.write(f"# t_max    = {t_max}\n")
18        f.write(f"# dx      = {dx}\n")
19        f.write(f"# dt      = {dt}\n")
20        f.write(f"# sigma_0  = {sigma_0}\n")
21        f.write(f"# k_0      = {k_0}\n")
22        f.write(f"# cach_deu_time = {save_every_time}\n")
23
24        f.write("\n" + "#" * 150 + "\n\n")
25
26        f.write(f"#{'t':>15s} {'x':>15s} {'Re_psi':>15s} {'Im_psi':>15s}
  ↳ {'|psi|':>15s} {'|psi|^2':>15s}\n")
27
28        for n in range(0, len(t), save_every_time):
29            for i in range(0, len(x), save_every_x):
30                psi = y[n, i]
31
32                f.write(f" {'t[n]:>15.6e} "
33                        f"{x[i]:>15.6e} "
34                        f"{psi.real:>15.6e} "
35                        f"{psi.imag:>15.6e} "
36                        f"{np.abs(psi):>15.6e} "
37                        f"{np.abs(psi)**2:>15.6e}\n")
38
39        f.write("\n")

```

Tên tập tin được ghép từ chuỗi TDSE-ketqua-, tham số `output_name` và đuôi `.txt`. Phần đầu tập tin lưu lại kích thước lưới, miền tính, bước lưới và tham số bó sóng; nhờ đó mỗi tập dữ liệu vẫn mang thông tin về phép chạy đã tạo ra nó.

Dòng tiêu đề gồm sáu cột: t , x , $\text{Re}\psi$, $\text{Im}\psi$, $|\psi|$ và $|\psi|^2$. Vòng ngoài duyệt thời gian với bước `save_every_time`, còn vòng trong duyệt không gian với bước `save_every_x`. Biến `psi=y[n,i]` lấy một phần tử phức rồi tách các đại lượng cần ghi. Định dạng 15.6e lưu số theo ký hiệu khoa học với sáu chữ số sau dấu thập phân. Dòng trống sau mỗi bước thời gian phân tách các khối dữ liệu, thuận tiện cho lệnh vẽ mặt bằng gnuplot. Hàm này được dùng chung cho cả RK4 và Leapfrog vì cấu trúc đầu ra [thời gian, không gian] giống nhau.

4.7. Tính và ghi chuẩn xác suất

```

1 def luu_file_xac_xuat(output_name, x, t, y, save_every_time=1):
2     global N_x, N_t, dx, dt, sigma_0, k_0, x_min, x_max, t_min, t_max
3     filename = f"TDSE-ketqua-{output_name}-xacxuat.txt"
4
5     with open(filename, "w", encoding="utf-8") as f:
6         f.write("# KET QUA GIAI PHUONG TRINH SCHRODINGER PHU THUOC THOI
7             ↪ GIAN\n\n")
8
9         f.write("#" * 150 + "\n")
10        f.write("# THAM SO DAU VAO\n")
11        f.write("#" * 150 + "\n\n")
12
13        f.write(f"# N_x      = {N_x}\n")
14        f.write(f"# N_t      = {N_t}\n")
15        f.write(f"# x_min    = {x_min}\n")
16        f.write(f"# x_max    = {x_max}\n")
17        f.write(f"# t_min    = {t_min}\n")
18        f.write(f"# t_max    = {t_max}\n")
19        f.write(f"# dx      = {dx}\n")
20        f.write(f"# dt      = {dt}\n")
21        f.write(f"# sigma_0  = {sigma_0}\n")
22        f.write(f"# k_0      = {k_0}\n")
23        f.write(f"# cach_deu_time = {save_every_time}\n")
24
25        f.write("\n" + "#" * 150 + "\n\n")
26
27        f.write(f"#{'t':>15s} {'Integral_ |psi|^2_dx':>25s}\n")
28
29        for n in range(0, len(t), save_every_time):
30
31            prob_x = np.abs(y[n, :])**2
32            tong_xac_xuat = np.trapz(prob_x, x)
33
34            f.write(f" {'t[n]:>15.6e} "
35                f"{'tong_xac_xuat:>25.6e}\n")

```

Hàm thứ hai tạo tập tin có hậu tố `-xacxuat.txt`. Tại mỗi thời điểm được chọn, câu lệnh `np.abs(y[n,:])**2` tạo mật độ xác suất trên toàn bộ lưới. `np.trapz(prob_x,x)` thực hiện

tích phân hình thang

$$N(t_n) \approx \int_{x_{\min}}^{x_{\max}} |\psi(x, t_n)|^2 dx.$$

4.8. Gán tham số, chạy hàm giải và lưu kết quả

```

1 sigma_0 = 0.5
2 dx = 0.05
3 dt = 1/4*dx**2
4 k_0 = 8
5 x_0 = 5
6
7 hbar = 1
8 m = 1
9
10 t_min = 0
11 t_max = 1.2
12
13 x_min = 0
14 x_max = 15
15
16 V_max = 500
17 kiểmtra_CFL(dt, dx, V_max)
18
19 x, t, N_x, N_t = khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt)
20 x, t, N_x, N_t = khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt)
21
22 psi_t_x_RK4 = giaipt_psi_RK4()
23
24 luu_ketqua_psi("RK4",x,t,y=psi_t_x_RK4,save_every_time=100)
25
26 luu_file_xac_xuat("RK4",x,t,y=psi_t_x_RK4,save_every_time=100)

```

Khối lệnh này xác lập cấu hình mô phỏng chính: $\sigma_0 = 0.5$, $k_0 = 8$, $x_0 = 5$, $\Delta x = 0.05$, $\Delta t = \Delta x^2/4$, $\hbar = m = 1$, miền $x \in [0, 15]$, miền $t \in [0, 1.2]$ và $V_{\max} = 500$ – hoàn toàn giống bộ tham số đã dùng cho Leapfrog, để hai phương pháp có thể so sánh trực tiếp.

Sau lệnh kiểm tra CFL, lưới được tạo (hai lần liên tiếp, lần thứ hai chỉ ghi đè lên kết quả của lần thứ nhất và không ảnh hưởng đến kết quả cuối). Hàm `giaipt_psi_RK4()` trả về toàn bộ lịch sử nghiệm `psi_t_x_RK4`, sau đó được ghi ra hai tập tin `TDSE-ketqua-RK4.txt` và `TDSE-ketqua-RK4-xacxuat.txt`.

4.9. Đọc dữ liệu để vẽ hình

```

1 import matplotlib.pyplot as plt
2 plt.style.use('sci.mplstyle')
3 t_xacsuat_RK4, N_RK4 = np.loadtxt("TDSE-ketqua-RK4-xacxuat.txt", unpack =
  ↳ True, comments = "#")
4 t_xacsuat_bac4, N_bac4 = np.loadtxt("TDSE-ketqua-Leapfrog-Bac4-xacxuat.txt",
  ↳ unpack = True, comments = "#")

```

```

5
6 x_bac2, t_bac2, psi_bac2 = np.loadtxt("TDSE-ketqua-RK4.txt", unpack = True,
  ↳ comments = "#", usecols=(0,1,4))
7
8 plt.figure(figsize=(7, 6))
9
10 fig, ax = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

```

Hai chuẩn xác suất $N(t)$ được đọc từ tập tin của RK4 (TDSE-ketqua-RK4-xacxuat.txt) và của Leapfrog bậc 4 (TDSE-ketqua-Leapfrog-Bac4-xacxuat.txt), để hai đường $N(t)$ có thể vẽ cạnh nhau trên cùng một hình. Dòng `x_bac2, t_bac2, psi_bac2 = np.loadtxt(...)` đọc thêm dữ liệu $|\psi(x, t)|$ từ TDSE-ketqua-RK4.txt; dữ liệu này phục vụ cho việc dựng hình tiến hóa $|\psi(x, t)|$ của RK4 (Hình 1b ở mục 5.1), không tham gia trực tiếp vào đồ thị bảo toàn xác suất.

4.10. Vẽ và lưu đồ thị bảo toàn xác suất

```

1 ax[0].plot(t_xacsuat_RK4, N_RK4, linewidth = 3)
2 ax[0].set_xlabel(r"$t$")
3 ax[0].set_ylabel(r"$N(t)$")
4 ax[0].set_title("Runge Kutta 4")
5 ax[0].grid(True, alpha=0.3)
6
7 ax[1].plot(t_xacsuat_bac4, N_bac4, linewidth = 3)
8 ax[1].set_xlabel(r"$t$")
9 ax[1].set_title("Leapfrog bac 4")
10 ax[1].grid(True, alpha=0.3)
11
12 ax[0].set_ylim(0,2)
13
14 ax[0].set_xlim(0,1)
15 ax[1].set_xlim(0,1)
16
17 plt.tight_layout()
18 plt.savefig("baotoan_xacsuat.pdf", bbox_inches="tight")
19 plt.show()

```

Hai ô con (`ax[0]` và `ax[1]`) dùng chung trục tung (`sharey=True`) và chung giới hạn $[0, 2]$, cho phép so sánh trực tiếp mức độ trôi của $N(t)$ giữa RK4 và Leapfrog bậc 4 trên cùng một thang đo. Tiêu đề của hai ô, "Runge Kutta 4" và "Leapfrog bac 4", xác định rõ phương pháp nào ứng với đường nào.

4.11. Tạo các giá trị σ_0 và k_0

```

1 sigma_0_list = [0.5, 1, 2]
2 k_0_list = [2, 8, 10, -2, -8, -10]
3
4 for sigma_value in sigma_0_list:
5     for k_value in k_0_list:

```

```

6      sigma_0 = sigma_value
7      k_0 = k_value
8      filename = (f"RK4_sigma={sigma_0}_k0={k_0}")
9
10     psi_t_x_bac_4 = giaipt_psi_RK4()
11
12     luu_ketqua_psi(
13         filename,
14         x,
15         t,
16         y=psi_t_x_bac_4,
17         save_every_time=100
18     )
19
20     luu_file_xac_xuat(
21         filename,
22         x,
23         t,
24         y=psi_t_x_bac_4,
25         save_every_time=100
26     )

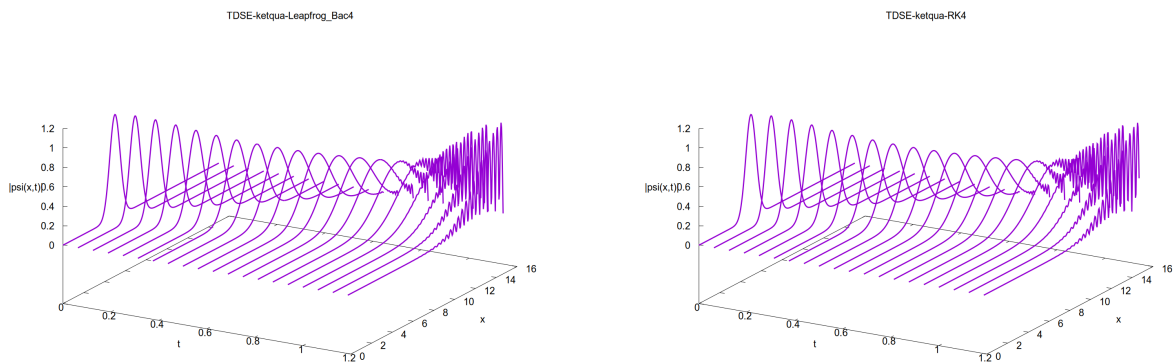
```

Hai danh sách tạo $3 \times 6 = 18$ tổ hợp tham số. Ở mỗi vòng lặp, các biến toàn cục `sigma_0` và `k_0` được cập nhật trước khi gọi `giaipt_psi_RK4()`, nên hàm `khointao_psi0` bên trong bộ giải tạo đúng bó sóng mới. Ma trận M , lưới, bước thời gian và các hằng số vật lý vẫn giữ nguyên – chỉ điều kiện đầu thay đổi theo σ_0 và k_0 .

5. Kết quả và thảo luận

5.1. So sánh Leapfrog bậc 4 và RK4

Hai hình dưới đây được tạo với cùng bộ tham số $\sigma_0 = 0.5$, $k_0 = 8$, $x_0 = 5$, $\Delta x = 0.05$ và $\Delta t = 0.000625$, chỉ khác cách rời rạc hóa thời gian.



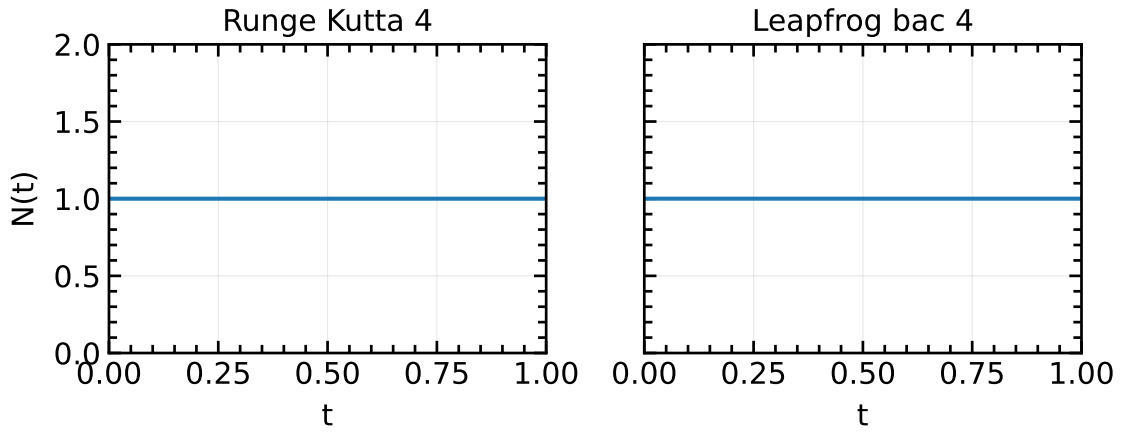
(a) Leapfrog với sai phân không gian bậc 4.

(b) RK4 với sai phân không gian bậc 4.

Hình 1: Sự tiến hóa của $|\psi(x, t)|$ khi $\sigma_0 = 0.5$ và $k_0 = 8$.

Ở cả hai kết quả, bó sóng xuất phát từ $x_0 = 5$, truyền theo chiều $+x$ với vận tốc nhóm $v_g = 8$ và giãn rộng theo thời gian. Phần đuôi bó sóng chạm tường $x = 15$ trước tâm, sau đó phản xạ ngược lại và giao thoa với sóng tới, tạo thành các vân gần biên. Vì hai phương pháp dùng cùng sai phân không gian bậc 4, hình dạng tổng thể của $|\psi(x, t)|$ rất giống nhau; sai khác nhỏ giữa Hình 1a và Hình 1b chủ yếu do bậc chính xác thời gian khác nhau ($\mathcal{O}(\Delta t^2)$ với Leapfrog so với $\mathcal{O}(\Delta t^4)$ với RK4) và do RK4 có miền động lực học rộng hơn một nút ở mỗi biên như đã nêu ở mục 3.2.

5.2. Bảo toàn xác suất



Hình 2: Chuẩn xác suất $N(t)$ của RK4 và Leapfrog bậc 4. Hai ô dùng chung giới hạn trục tung.

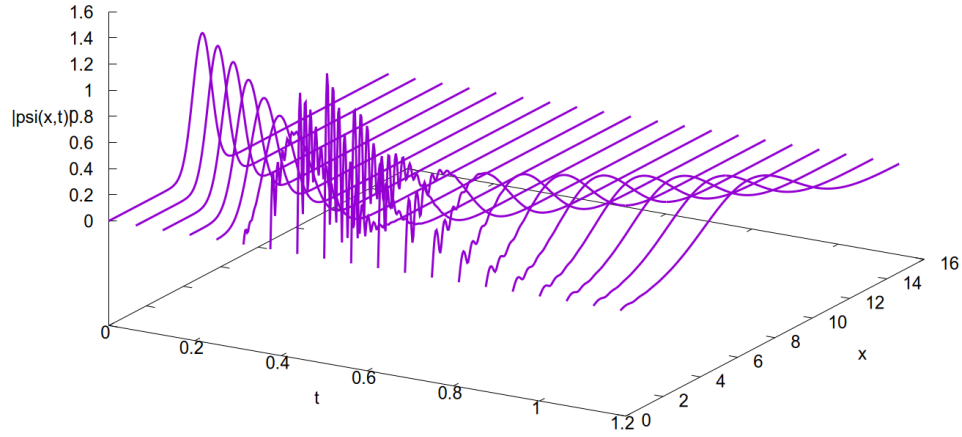
Tại $t = 0$, hệ số chuẩn hóa của bó Gauss cho $N(0) \simeq 1$. Các giá trị thu được xấp xỉ:

t	$N_{\text{RK4}}(t)$	$N_{\text{Leapfrog bậc 4}}(t)$
0	1.000000	1.000000
0.0625	1.000000	1.059116
0.5000	1.000000	1.062457
1.0000	1.000000	1.062448

Với RK4, $N(t)$ giữ nguyên giá trị 1.000000 (sai khác chỉ ở bậc 10^{-9}) trên toàn bộ khoảng thời gian mô phỏng. Ngược lại, với Leapfrog bậc 4, $N(t)$ tăng đột ngột từ 1 lên khoảng 1.0591 ngay sau bước thời gian đầu tiên (bước Euler khởi động), rồi tiếp tục trôi nhẹ đến khoảng 1.0625 trước khi gần như không đổi. Sự khác biệt này phản ánh trực tiếp bậc chính xác thời gian: sai số toàn cục $\mathcal{O}(\Delta t^4)$ của RK4 nhỏ hơn rất nhiều so với $\mathcal{O}(\Delta t^2)$ của Leapfrog với cùng $\Delta t = 6.25 \times 10^{-4}$, nên RK4 bảo toàn xác suất tốt hơn đáng kể.

5.3. Sóng truyền ngược khi $k_0 = -10$

TDSE-ketqua-RK4_sigma=0.5_k0=-10



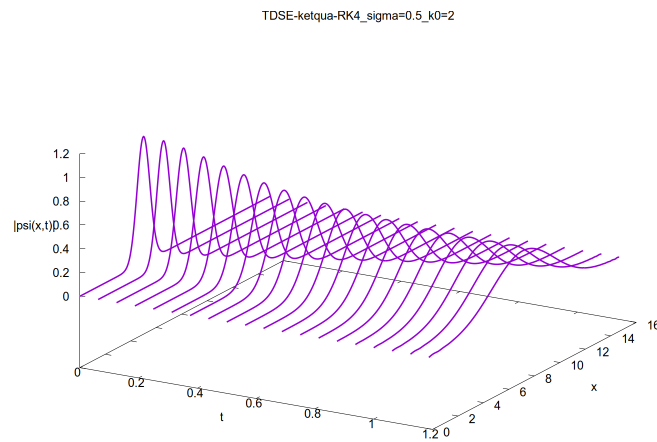
Hình 3: RK4 với $\sigma_0 = 0.5$ và $k_0 = -10$: bó sóng truyền về phía $-x$, phản xạ ở biên trái rồi đổi chiều.

Với $k_0 = -10$, vận tốc nhóm ban đầu là $v_g = -10$. Tâm bó sóng cách biên trái một đoạn 5, nên thời điểm va chạm ước lượng là

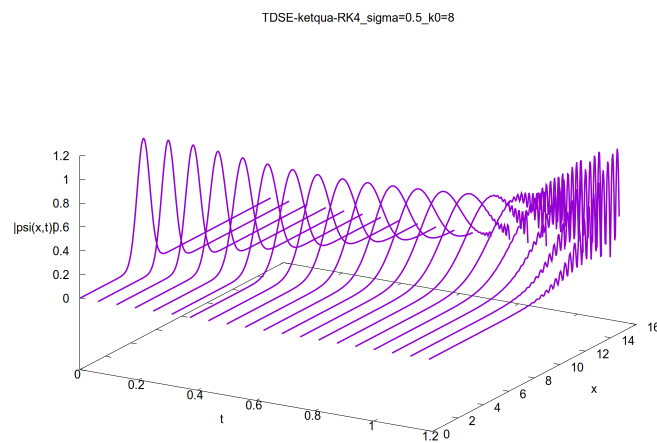
$$t \approx \frac{5}{10} = 0.5. \quad (23)$$

Trước thời điểm này, đỉnh xác suất dịch về trái. Khi thành phần tới chồng lên thành phần phản xạ, $|\psi|$ xuất hiện hiện tượng giao thoa. Sau khi phản xạ hoàn tất, bó sóng chuyển động về phía $+x$.

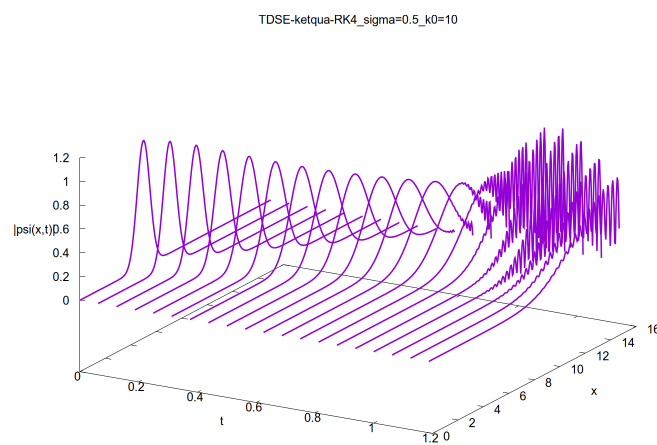
5.4. Ảnh hưởng của độ lớn k_0 đến vận tốc và phản xạ



(a) $k_0 = 2$.



(b) $k_0 = 8$.



(c) $k_0 = 10$.

Hình 4: Sóng truyền về phía $+x$ với cùng $\sigma_0 = 0.5$ và ba giá trị số sóng trung tâm.

Các thời điểm để tâm bó sóng đi từ $x_0 = 5$ đến biên phải $x = 15$ theo vận tốc nhóm liên tục là

$$t \approx \frac{15 - 5}{k_0}. \quad (24)$$

Suy ra:

k_0	v_g	t ước lượng
2	2	5.0
8	8	1.25
10	10	1.0

Với $k_0 = 2$, thời gian mô phỏng 1.2 ngắn hơn nhiều so với thời gian chạm biên. Hình 4a vì vậy chủ yếu thể hiện chuyển động tịnh tiến và sự giãn rộng của bó sóng, chưa hình thành phản xạ rõ rệt.

Với $k_0 = 8$, tâm bó sóng chưa hoàn toàn đạt biên phải tại $t = 1.2$, nhưng phần đuôi đã tương tác với vùng tường. Hình 4b cho phép quan sát giai đoạn bắt đầu chồng chập giữa sóng tới và sóng phản xạ.

Với $k_0 = 10$, tâm bó sóng đạt biên vào khoảng $t = 1$. Khoảng thời gian còn lại đủ để thành phần phản xạ quay trở lại miền trong. Trong vùng hai thành phần cùng tồn tại, mật độ xác suất có các cực đại và cực tiểu liên tiếp. Đây là các vân giao thoa do tổng biên độ

$$\psi(x, t) = \psi_{\text{tới}}(x, t) + \psi_{\text{phản xạ}}(x, t) \quad (25)$$

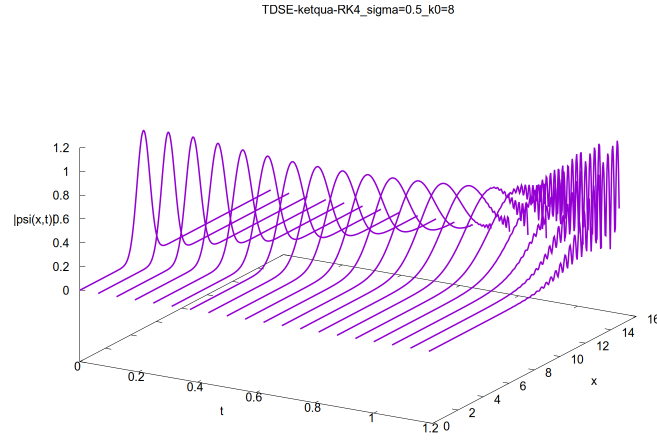
thỏa

$$|\psi|^2 = |\psi_{\text{tới}}|^2 + |\psi_{\text{phản xạ}}|^2 + 2 \operatorname{Re}(\psi_{\text{tới}}^* \psi_{\text{phản xạ}}). \quad (26)$$

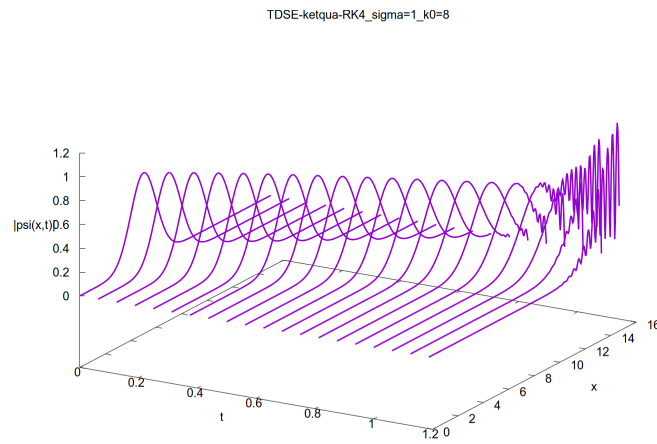
Số vân tăng khi $|k_0|$ lớn vì bước sóng de Broglie $\lambda = 2\pi/|k_0|$ giảm.

5.5. Ảnh hưởng của độ rộng ban đầu σ_0

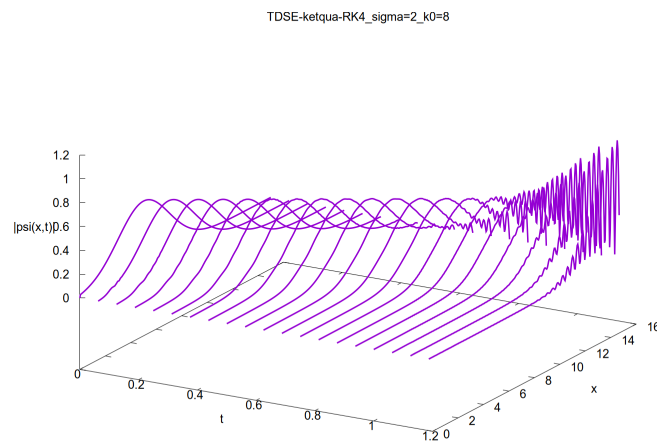
Để khảo sát riêng ảnh hưởng của độ rộng bó sóng, số sóng trung tâm được giữ cố định tại $k_0 = 8$, trong khi σ_0 lần lượt nhận các giá trị 0.5, 1 và 2. Kết quả mô phỏng bằng phương pháp RK4 được trình bày trong Hình 5.



(a) $\sigma_0 = 0.5$.



(b) $\sigma_0 = 1$.



(c) $\sigma_0 = 2$.

Hình 5: Sự tiến hóa của bó sóng khi giữ cố định $k_0 = 8$ và thay đổi độ rộng ban đầu σ_0 .

Theo Phương trình (4), σ_0 xác định độ rộng ban đầu của bó sóng. Khi σ_0 tăng, bó sóng

rộng hơn trong không gian nhưng phân bố số sóng hẹp hơn, nên mức độ giãn theo thời gian giảm.

Với $\sigma_0 = 0.5$ trong Hình 5a, bó sóng hẹp và giãn rõ nhất. Khi $\sigma_0 = 1$ trong Hình 5b, bó sóng rộng hơn và giãn ít hơn. Với $\sigma_0 = 2$ trong Hình 5c, bó sóng rộng ngay từ đầu và giãn ít nhất, nhưng phần đuôi có thể chạm biên sớm hơn.

Do $k_0 = 8$ được giữ cố định trong cả ba trường hợp, vận tốc nhóm và quỹ đạo của tâm bó sóng gần như không đổi. Sau khi gặp biên, sóng phản xạ chồng chập với sóng tới và tạo thành các vân giao thoa.

6. Kết luận

Trong bài này, TDSE được rời rạc hóa theo không gian bằng sai phân trung tâm bậc 4 (5 điểm), đưa về hệ ODE tuyến tính $d\psi/dt = M\psi$, và được tiến theo thời gian bằng phương pháp Runge-Kutta bậc 4. Khác với Leapfrog, RK4 là phương pháp một bước nên không cần công thức cho bước thời gian đầu tiên.

Với $\sigma_0 = 0.5$ và $k_0 = 8$, RK4 cho hình dạng tiến hóa $|\psi(x, t)|$ tương tự Leapfrog bậc 4: bó sóng truyền sang phải, giãn rộng và phản xạ tại biên. Tuy nhiên, chuẩn xác suất của RK4 giữ nguyên ở 1.000000 trong suốt mô phỏng, trong khi Leapfrog bậc 4 tăng từ 1 lên khoảng 1.0625 ngay từ bước đầu. Sự khác biệt này phù hợp với bậc chính xác thời gian cao hơn của RK4 ($\mathcal{O}(\Delta t^4)$ so với $\mathcal{O}(\Delta t^2)$).

Các kết quả với $k_0 = -10, 2, 8, 10$ cho thấy dấu của k_0 quyết định hướng truyền, còn $|k_0|$ càng lớn thì bó sóng chạm biên và xuất hiện giao thoa càng sớm – kết luận này không phụ thuộc vào việc dùng RK4 hay Leapfrog, vì cả hai cùng mô tả đúng động lực học của bó sóng với sai phân không gian bậc 4.